

Mangalayan University Online
Assignment Cover Page- July 23 Session

Maximum Marks: 30,

Last Date of Submission: 20 Jan'25

Learner Name: Amit kumar

Registration No: 22303265

Course Name: Master of Computer Application(MCA)

Assignment: Java Programming Lab-CSM-7151

Date of Submission: 05/01/2025

Note- The assignment Question have 5 Sections/Blocks. Kindly attempt Any ONE question from Each of the 5 blocks. Each question carries equal marks. For Eg:

Block 1 Attempt- Question 1a or 1b

Block 2 Attempt- Question 2a or 2b

Block 3 Attempt- Question 3a or 3b

Block 4 Attempt- Question 4a or 4b

Block 5 Attempt- Question 5a or 5b

.....
....

Block 1 - Q 1B

Write a Java program to count the number of vowels and consonants in a string.

Ans. Here's a simple Java program that counts the number of vowels and consonants in a given string:

```
import java.util.Scanner;
```

```
public class VowelConsonantCounter {
```

```
    public static void main(String[] args) {  
        // Create a Scanner object to read input
```

```

Scanner scanner = new Scanner(System.in);

// Prompt the user to enter a string
System.out.print("Enter a string: ");
String input = scanner.nextLine();

// Convert the string to lowercase to handle case insensitivity
input = input.toLowerCase();

// Initialize counters for vowels and consonants
int vowelCount = 0;
int consonantCount = 0;

// Loop through each character in the string
for (int i = 0; i < input.length(); i++) {
    char ch = input.charAt(i);

    // Check if the character is a letter
    if (Character.isLetter(ch)) {
        // Check if the character is a vowel
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
            vowelCount++;
        } else {
            consonantCount++;
        }
    }
}

// Output the results
System.out.println("Number of vowels: " + vowelCount);
System.out.println("Number of consonants: " + consonantCount);

// Close the scanner
scanner.close();
}

```

How the program works:

1. The program prompts the user to input a string.

2. It converts the string to lowercase to ensure case-insensitive counting of vowels and consonants.
3. It iterates through each character of the string:
 - It checks if the character is a letter (ignores spaces, punctuation, and numbers).
 - If it's a vowel (a, e, i, o, u), it increments the `vowelCount`.
 - Otherwise, it counts it as a consonant and increments the `consonantCount`.
4. Finally, it prints the number of vowels and consonants in the string.

Example output:

Enter a string: Hello World!

Number of vowels: 3

Number of consonants: 7

This program should handle most common input scenarios, including mixed-case strings and non-alphabetical characters.

Block 2 - Q 2B

How do you handle exceptions in Java? Provide an example.

Ans. In Java, exceptions are handled using a mechanism called **try-catch blocks**. This mechanism allows you to catch and handle errors or exceptional situations that may occur during the execution of the program. Additionally, Java provides the `finally` block, which allows you to execute certain code regardless of whether an exception occurred or not.

Steps for Exception Handling:

1. **try block:** You place the code that may throw an exception inside the `try` block.
2. **catch block:** You catch and handle the exception using one or more `catch` blocks.
3. **finally block** (optional): This block executes code that should run regardless of whether an exception occurred, such as releasing resources.

Example of Exception Handling in Java:

Here is a simple example where we attempt to divide two numbers and handle a potential `ArithmeticException` (such as division by zero):

```
import java.util.Scanner;

public class ExceptionHandlingExample {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            // Ask the user to input two numbers
            System.out.print("Enter the numerator: ");
            int numerator = scanner.nextInt();

            System.out.print("Enter the denominator: ");
            int denominator = scanner.nextInt();

            // Attempt division
            int result = numerator / denominator;
            System.out.println("The result of division is: " + result);
        } catch (ArithmeticException e) {
            // Handle the case of division by zero
            System.out.println("Error: Cannot divide by zero.");
        } catch (Exception e) {
            // Catch any other unforeseen exception
            System.out.println("An error occurred: " + e.getMessage());
        } finally {
            // This block will always execute, whether exception occurs or not
            System.out.println("Execution completed.");
            scanner.close();
        }
    }
}
```

Explanation of the Example:

1. **try block:** The code inside the `try` block attempts to divide the two user-inputted numbers.
2. **catch blocks:**

- The first **catch** block specifically handles **ArithmeticException**, which occurs if the user tries to divide by zero.
 - The second **catch** block is a general exception handler that catches any other exceptions that might occur.
3. **finally block**: This block always executes, regardless of whether an exception was thrown or not. It can be used for cleanup tasks, like closing resources.

Sample Output:

If you input a valid division (e.g., 10 and 2):

Enter the numerator: 10
Enter the denominator: 2
The result of division is: 5
Execution completed.

•

If you try dividing by zero:

Enter the numerator: 10
Enter the denominator: 0
Error: Cannot divide by zero.
Execution completed.

•

If you input invalid data (e.g., non-integer input), it catches a general **Exception**:

Enter the numerator: 10
Enter the denominator: abc
An error occurred: java.util.InputMismatchException
Execution completed.

•

Key Points:

- **Multiple catch blocks:** You can have multiple `catch` blocks to handle different types of exceptions (e.g., `ArithmeticException`, `InputMismatchException`, etc.).
- **Generic `Exception` catch:** Catching the generic `Exception` type can help catch unforeseen errors, but it's usually better to catch specific exceptions.
- **Finally block:** Used for cleaning up resources like closing files or database connections, regardless of whether an exception occurred.

This approach helps in making the program more robust and allows the developer to gracefully handle errors instead of crashing the application.

Block 3 - Q 3B

What is inheritance in Java? Give an example. What is polymorphism in Java? Provide an example.

Ans. Inheritance in Java:

Inheritance is one of the fundamental concepts of Object-Oriented Programming (OOP). It allows one class (the **subclass** or **child class**) to inherit the fields and methods of another class (the **superclass** or **parent class**). This helps promote code reusability and establishes a relationship between classes.

In Java, inheritance is achieved using the `extends` keyword.

Key Points about Inheritance:

- **Single Inheritance:** A class can inherit from only one class in Java (Java does not support multiple inheritance for classes).
- **Method Overriding:** A subclass can override methods of its superclass to provide its specific implementation.
- **Access to Superclass Members:** The subclass inherits non-private members (fields and methods) from the superclass.

Example of Inheritance in Java:

```
// Parent class (Superclass)
class Animal {
    // Field
```

```
String name;

// Constructor
public Animal(String name) {
    this.name = name;
}

// Method
public void makeSound() {
    System.out.println("Some animal sound");
}
}

// Child class (Subclass) that inherits from Animal
class Dog extends Animal {

    // Constructor
    public Dog(String name) {
        super(name); // Call to the superclass constructor
    }

    // Method overriding
    @Override
    public void makeSound() {
        System.out.println(name + " barks");
    }
}

public class InheritanceExample {
    public static void main(String[] args) {
        Animal animal = new Animal("Generic Animal");
        Dog dog = new Dog("Buddy");

        // Calling method from parent class
        animal.makeSound();

        // Calling overridden method from child class
        dog.makeSound();
    }
}
```

Output:

Some animal sound
Buddy barks

Explanation:

- The class **Dog** **inherits** from the **Animal** class.
- The **Dog** class **overrides** the **makeSound()** method to provide its specific implementation.
- The **super()** keyword is used to call the constructor of the superclass (**Animal**).

Polymorphism in Java:

Polymorphism is another core concept in OOP that means "many forms." It allows an object to take many forms, which can be achieved in two main ways:

1. **Method Overloading** (Compile-time Polymorphism)
2. **Method Overriding** (Runtime Polymorphism)

Polymorphism enables a single interface to represent different underlying forms (objects or methods).

Types of Polymorphism:

- **Compile-time (Static) Polymorphism:** Occurs when multiple methods with the same name exist but have different signatures (e.g., different parameters or return types). This is known as method overloading.
- **Runtime (Dynamic) Polymorphism:** Occurs when a subclass overrides a method of the superclass, and the method is called on a reference of the superclass type, which points to a subclass object. This is known as method overriding.

Example of Polymorphism in Java:

Method Overloading (Compile-time Polymorphism):

```
class Calculator {  
    // Method to add two integers
```



```

    public int add(int a, int b) {
        return a + b;
    }

    // Overloaded method to add three integers
    public int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class PolymorphismExample {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println("Sum of 2 numbers: " + calc.add(5, 10));    // Calls
add(int, int)
        System.out.println("Sum of 3 numbers: " + calc.add(5, 10, 15)); // Calls
add(int, int, int)
    }
}

```

Output:

```

Sum of 2 numbers: 15
Sum of 3 numbers: 30

```

Method Overriding (Runtime Polymorphism):

```

// Parent class
class Shape {
    public void draw() {
        System.out.println("Drawing a Shape");
    }
}

// Child class (Rectangle)
class Rectangle extends Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

```

```

}

// Child class (Circle)
class Circle extends Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a Circle");
    }
}

public class PolymorphismExample {
    public static void main(String[] args) {
        // Polymorphic behavior
        Shape shape1 = new Rectangle(); // Shape reference, Rectangle object
        Shape shape2 = new Circle();    // Shape reference, Circle object

        shape1.draw(); // Calls Rectangle's draw method
        shape2.draw(); // Calls Circle's draw method
    }
}

```

Output:

```

Drawing a Rectangle
Drawing a Circle

```

Explanation:

- **Method Overloading:** The `add` method is overloaded with different numbers of parameters (two and three). The compiler decides which method to call based on the method signature at compile time.
- **Method Overriding:** The `draw` method is overridden in the `Rectangle` and `Circle` subclasses. At runtime, the correct method is invoked based on the actual object type (either `Rectangle` or `Circle`), even though the reference is of type `Shape`.

Summary:

- **Inheritance** allows a subclass to inherit fields and methods from a superclass, promoting code reusability.
- **Polymorphism** enables an object to take many forms, achieved via method overloading (compile-time) or method overriding (runtime).

Block 4 - Q 4A

What is the difference between ArrayList and LinkedList in Java?

Ans. In Java, both **ArrayList** and **LinkedList** are part of the **List** interface, providing a way to store and manipulate a collection of elements. Despite their shared interface, they differ significantly in their internal structure, performance characteristics, and use cases. Here's a detailed comparison:

1. Underlying Data Structure:

- **ArrayList:**
 - It uses a dynamic array to store elements. The elements are stored in contiguous memory locations, and the array size grows as elements are added.
- **LinkedList:**
 - It uses a doubly linked list to store elements. Each element (node) contains references (pointers) to both the previous and next elements in the list.

2. Performance Characteristics:

- **ArrayList:**
 - **Access Time (get):** The time complexity for accessing an element by index is **O(1)**, as elements are stored contiguously and can be directly accessed using their index.
 - **Insertions/Deletions:** Inserting or removing elements at the **end** is **O(1)**. However, inserting or removing elements at the **beginning or middle** requires shifting elements, which has a time complexity of **O(n)**.
 - **Memory Usage:** **ArrayList** requires less memory compared to **LinkedList**, as it only stores the elements themselves, without the need for additional references to the previous and next nodes.

- **LinkedList:**
 - **Access Time (get):** The time complexity for accessing an element by index is $O(n)$. This is because to access an element, you need to traverse the list starting from either the head or the tail, depending on which is closer.
 - **Insertions/Deletions:** Inserting or removing elements at the **beginning** or **end** is $O(1)$, as the list only requires updating the references in the adjacent nodes. However, for insertion or removal in the **middle**, the time complexity is $O(n)$, as the list must be traversed to the correct position.
 - **Memory Usage:** **LinkedList** uses more memory because each node must store references to both the previous and next elements, in addition to the element itself.

3. Use Cases:

- **ArrayList:** Best suited for situations where you need frequent access to elements by index. It's ideal when the number of insertions and deletions is low, especially when those operations happen at the end of the list.
- **LinkedList:** More efficient for use cases where you frequently insert or delete elements at the beginning or middle of the list. It's not optimal for random access but excels in scenarios where the list changes frequently.

4. Resizing:

- **ArrayList:** The underlying array must be resized (copied to a new array) when the list exceeds its capacity, which can lead to performance overhead during resizing.
- **LinkedList:** Does not require resizing, as elements are linked dynamically.

Conclusion:

- Use **ArrayList** when you need fast access to elements by index and minimal insertions/removals.
- Use **LinkedList** when your use case involves frequent insertions and deletions, especially at the start or middle of the list.

Block 5 - Q 5B

Write a Java program to implement a simple "Student" class using constructors. What is an abstract class in Java? Provide an example. What is the use of the super keyword in Java?

Ans. 1. Java Program to Implement a Simple "Student" Class Using Constructors:

Below is a Java program that implements a **Student** class using constructors:

```
// Student class with attributes and constructors
class Student {
    // Instance variables
    private String name;
    private int age;
    private int rollNumber;

    // Constructor with parameters to initialize the student object
    public Student(String name, int age, int rollNumber) {
        this.name = name;
        this.age = age;
        this.rollNumber = rollNumber;
    }

    // Constructor with default values
    public Student() {
        this.name = "Unknown";
        this.age = 0;
        this.rollNumber = 0;
    }

    // Method to display student details
    public void displayDetails() {
        System.out.println("Student Name: " + name);
        System.out.println("Student Age: " + age);
        System.out.println("Student Roll Number: " + rollNumber);
    }
}
```

```
public class StudentExample {  
    public static void main(String[] args) {  
        // Creating Student objects using both constructors  
        Student student1 = new Student("John Doe", 20, 101);  
        Student student2 = new Student(); // Default constructor  
  
        // Display details of both students  
        student1.displayDetails();  
        System.out.println();  
        student2.displayDetails();  
    }  
}
```

Output:

Student Name: John Doe
Student Age: 20
Student Roll Number: 101

Student Name: Unknown
Student Age: 0
Student Roll Number: 0

Explanation:

1. The **Student** class has three instance variables: **name**, **age**, and **rollNumber**.
 2. There are two constructors:
 - One constructor initializes the **name**, **age**, and **rollNumber** with values provided when creating a **Student** object.
 - Another constructor provides default values when no arguments are passed.
 3. The **displayDetails()** method displays the values of the **Student** attributes.
 4. In the **main()** method, two **Student** objects are created: one using the parameterized constructor and the other using the default constructor.
-

2. Abstract Class in Java:

An **abstract class** in Java is a class that cannot be instantiated on its own and is meant to be subclassed. It can have both abstract methods (methods without implementation) and concrete methods (methods with implementation). Subclasses that extend an abstract class must provide implementations for all abstract methods.

Key Points:

- Abstract classes can have abstract and non-abstract methods.
- An abstract class cannot be instantiated directly.
- It is used when you want to provide common functionality for subclasses but leave certain methods to be defined by the subclasses.

Example of an Abstract Class in Java:

```
// Abstract class
abstract class Animal {
    // Abstract method (no implementation)
    public abstract void sound();

    // Regular method with implementation
    public void sleep() {
        System.out.println("The animal is sleeping.");
    }
}

// Subclass (inherit from Animal)
class Dog extends Animal {
    // Providing implementation for the abstract method
    public void sound() {
        System.out.println("The dog barks.");
    }
}

public class AbstractClassExample {
    public static void main(String[] args) {
        // Cannot instantiate Animal directly, as it's abstract
        // Animal animal = new Animal(); // Error!

        // Create an object of Dog (concrete class)
```

```
Animal dog = new Dog();
dog.sound(); // Calls the Dog's implementation of sound()
dog.sleep(); // Calls the common sleep() method from Animal
    }
}
```

Output:

The dog barks.

The animal is sleeping.

Explanation:

- The `Animal` class is abstract and contains an abstract method `sound()`, which must be implemented by any subclass.
 - The `Dog` class extends `Animal` and provides an implementation for the `sound()` method.
 - An object of `Dog` is created, and both `sound()` and `sleep()` methods are called.
-

3. Use of the `super` Keyword in Java:

The `super` keyword in Java refers to the superclass of the current object. It is used to access superclass methods and constructors. Here are the main uses of `super`:

1. **Accessing superclass methods:** It allows the subclass to invoke methods of the superclass, including overridden methods.
2. **Accessing superclass constructors:** It is used to call the constructor of the superclass from the subclass constructor.
3. **Accessing superclass fields:** It can be used to refer to fields in the superclass if they are hidden by fields in the subclass.

Example of `super` Keyword Usage:

```
class Vehicle {
    String type;

    // Constructor of superclass
```



```

public Vehicle(String type) {
    this.type = type;
}

public void display() {
    System.out.println("This is a " + type);
}
}

class Car extends Vehicle {
    String brand;

    // Constructor of subclass, calling superclass constructor using super
    public Car(String type, String brand) {
        super(type); // Call to Vehicle constructor
        this.brand = brand;
    }

    // Method overriding
    public void display() {
        super.display(); // Calling the superclass method
        System.out.println("Brand: " + brand);
    }
}

public class SuperKeywordExample {
    public static void main(String[] args) {
        // Creating a Car object
        Car car = new Car("Vehicle", "Toyota");

        // Calling the overridden display method
        car.display();
    }
}

```

Output:

```

This is a Vehicle
Brand: Toyota

```

Explanation:

- The `super` keyword is used in the `Car` class constructor to call the constructor of the `Vehicle` class.
- In the `display()` method, `super.display()` is used to invoke the `display()` method from the `Vehicle` class before adding the subclass-specific behavior.

Summary of the `super` Keyword:

- `super()`: Used to call the superclass constructor.
- `super.method()`: Used to call a method of the superclass.
- `super.variable`: Used to access a superclass field if it is hidden by a subclass field.